

the gradient. In order to minimize, we move in the opposite direction of the gradient. This means we can alter our original gradient ascent code as in [Listing 3-3](#).

```
threshold = 0.0001
maximum_iterations = 10000

def revenue_derivative_flipped(tax):
    return(0-revenue_derivative(tax))

current_rate = 0.7

keep_going = True
iterations = 0
while(keep_going):
    rate_change = step_size * revenue_derivative_flipped(current_rate)
    current_rate = current_rate - rate_change
    if(abs(rate_change) < threshold):
        keep_going = False
    if(iterations >= maximum_iterations):
        keep_going = False
    iterations = iterations + 1
```

[*Listing 3-3: Implementating gradient descent*](#)

Here everything is the same except we have changed a + to a - when we change the `current_rate`. By making this very small change, we've converted gradient ascent code to gradient descent code. In a way, they're essentially the same thing; they use a gradient to determine a direction, and then they move in that direction toward a definite goal. In fact, the most common convention today is to speak of gradient descent, and to refer to gradient ascent as a slightly altered version of gradient descent, the opposite of how this chapter has introduced it.

Hill Climbing in General

Being elected prime minister is a rare occurrence, and setting taxation rates to maximize government revenue is not an everyday activity even for prime ministers. (For the real-life version of the taxation/revenue discussion at the beginning of the chapter, I encourage you to look up the Laffer curve.)